Région Hauts-de-France
Cofinancé par l'Union européenne
realconseil
DIAGNOSTIC ACCOMPAGNEMENT FORMATION

SYMFONY 6 – Les firewall & la sécurité du loginForm

Introduction

Nous avons précédemment hashé le mot de passe et nous avons créé nos user, nous allons nous occuper du formLogin et sa sécurité et nous allons voir ici les firewall

I) Les Firewalls

Nous allons comme bien souvent, aller sur la documentation de symfony en ce qui concerne les firewall. C'est la section la plus importante en terme de sécurité, il va définir les sections de notre application qui sont sécurisés et de définir la façon d'authentification, ici un formulaire de connexion.

Allons voir notre fichier securtiy.yaml :

```
4        Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface: 'a
5    # https://symfony.com/doc/current/security.html#loading-the-user-the-user-provi
6    providers:
7        # used to reload user from session & other features (e.g. switch_user)
8        app_user_provider:
9            entity:
10               class: App\Entity\User
11               property: email
12   firewalls:
13       dev:
14           pattern: ^/(_(profiler|wdt)|css|images|js)/
15           security: false
16       main:
17           lazy: true
18           provider: app_user_provider
19
20           # activate different ways to authenticate
21           # https://symfony.com/doc/current/security.html#the-firewall
22
23           # https://symfony.com/doc/current/security/impersonating_user.html
24           # switch_user: true
```

LIEN DU PROVIDER

Nous avons de base un lien du provider qui est défini.

- Le provider

  - Dans Symfony 6, un "provider" fait généralement référence à une classe qui fournit des services au sein de l'application. Les services sont des objets qui effectuent des tâches spécifiques et peuvent être utilisés dans différentes parties de votre application Symfony.
  - Un provider est une façon de configurer et de définir des services dans Symfony. Il peut être considéré comme une classe responsable de l'instanciation et de la configuration des services. Il est généralement utilisé pour regrouper des services liés dans une même classe afin de simplifier leur gestion.

- Un provider est généralement une classe PHP qui implémente l'interface ServiceProviderInterface. Cette interface définit une méthode register() dans laquelle vous pouvez définir les services à ajouter au conteneur de services de Symfony.
- Voici un exemple simple de ce à quoi pourrait ressembler un provider dans Symfony 6 :

```php
use Symfony\Component\DependencyInjection\ContainerBuilder;
use Symfony\Component\DependencyInjection\ServiceProviderInterface;

class MyServiceProvider implements ServiceProviderInterface
{
    public function register(ContainerBuilder $container)
    {
        $container->register('my_service', MyService::class)
            ->addArgument('param1')
            ->addArgument('param2');
    }
}
```

(*PS : ceci est à titre d'exemple ne pas implémenter dans le projet*)

Dans cet exemple, MyServiceProvider est un provider qui enregistre un service appelé 'my_service'. Le service MyService est créé avec deux arguments (param1 et param2). Le conteneur de services de Symfony sera ensuite en mesure d'instancier MyService et d'injecter les arguments spécifiés lorsqu'il sera demandé.

Une fois que vous avez créé votre provider, vous devez l'enregistrer dans le fichier de configuration de votre application (par exemple config/services.php) pour qu'il soit pris en compte par Symfony et que les services soient disponibles.

Les providers sont un moyen puissant de gérer la configuration des services dans Symfony 6 et ils contribuent à la modularité et à la réutilisabilité de votre code en centralisant la configuration des services associés.

Revenons à notre firewall, et nous allons regarder de plus près notre formulaire de connexion

II) Le formulaire de connexion

Si nous allons voir la documentation, nous avons une commande toute faite pour faire notre formulaire de connexion.

Reprenons cette commande à faire dans notre terminal et analysons ce qui s'est produit

Nous avons donc la création de :

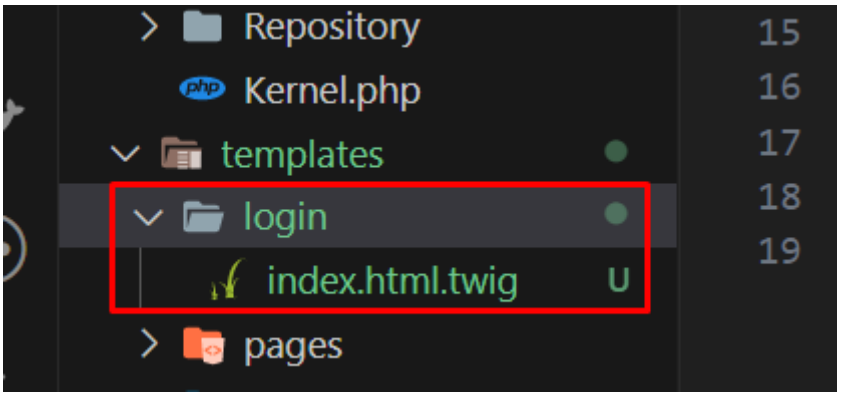
- Controller LoginController

```
src > Controller > LoginController.php > ...
<?php

namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;

class LoginController extends AbstractController
{
    #[Route('/login', name: 'app_login')]
    public function index(): Response
    {
        return $this->render('login/index.html.twig', [
            'controller_name' => 'LoginController',
        ]);
    }
}
```

Dans ce controller nous voyons la route et ce que cela nous renvoie qui est une view appelé login/index.html.twig

Essayons de voir si nous avons ça dans nos templates

- La view

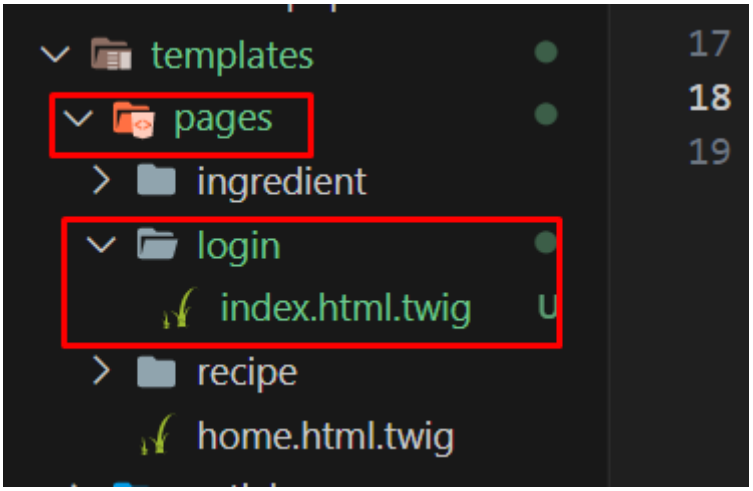


Nous avons bien le dossier spécifié et le fichier créé regardons de plus près ce fichier

```
templates > login > index.html.twig
1   {% extends 'base.html.twig' %}
2
3   {% block title %}Hello LoginController!{% endblock %}
4
5   {% block body %}
6   <style>
7       .example-wrapper { margin: 1em auto; max-width: 800px; width: 95%; font: 18px/1.5 sans-serif; }
8       .example-wrapper code { background: #F5F5F5; padding: 2px 6px; }
9   </style>
10
11  <div class="example-wrapper">
12      <h1>Hello {{ controller_name }}! ✅ </h1>
13
14      This friendly message is coming from:
15      <ul>
16          <li>Your controller at <code><a href="{{ 'C:/Users/alexa/OneDrive/Bureau/workspace/symfonytuto/first
17          <li>Your template at <code><a href="{{ 'C:/Users/alexa/OneDrive/Bureau/workspace/symfonytuto/firstpr
18      </ul>
19  </div>
20  {% endblock %}
21
```

Un fichier twig de base

Ce que nous allons faire dans un premier temps c'est de mettre ce dossier login dans le dossier pages et redéfinir le chemin dans le controller



Pour le chemin dans le controller :

```
13      {
14          return $this->render('pages/login/index.html.twig', [
15              'controller_name' => 'LoginController',
16          ]);
17      }
```

III) Le loginForm

Toujours dans la même documentation de symfony on nous précise de rajouter le login_path et le check_path dans le security.yaml regardons le notre et mettons le au niveau du firewall comme ceci

ATTENTION AUX INDENTATIONS

```
7           # used to reload user from session & other features (e.g. switch_user)
8           app_user_provider:
9               entity:
10                  class: App\Entity\User
11                  property: email
12      firewalls:
13          dev:
14              pattern: ^/(_(profiler|wdt)|css|images|js)/
15              security: false
16          main:                                          A AJOUTER
17              lazy: true
18              provider: app_user_provider
19              form_login:
20                  # "app_login" is the name of the route created previously
21                  login_path: app_login
22                  check_path: app_login      You, 1 second ago • Uncommitted changes
23
24              # activate different ways to authenticate
25              # https://symfony.com/doc/current/security.html#the-firewall
26
27              # https://symfony.com/doc/current/security/impersonating_user.html
```

Toujours par rapport à la documentation nous allons modifier notre loginController en suivant ce que l'on nous dit :

The User
- Loading the User: The User Provider
- Registering the User: Hashing Passwords
The Firewall
- Fetching the Firewall Configuration for a Request
Authenticating Users
- Form Login
- JSON Login
- HTTP Basic
- Login Link
- Access Tokens
- X.509 Client Certificates
- Remote Users
- Limiting Login Attempts
Login Programmatically
Logging Out
- Logout programmatically
- Customizing Logout
Fetching the User Object

Edit the login controller to render the login form:

```
// ...
+ use Symfony\Component\Security\Http\Authentication\AuthenticationUtils;

  class LoginController extends AbstractController
  {
      #[Route('/login', name: 'app_login')]
-     public function index(): Response
+     public function index(AuthenticationUtils $authenticationUtils): Response
      {
+         // get the login error if there is one
+         $error = $authenticationUtils->getLastAuthenticationError();
+
+         // last username entered by the user
+         $lastUsername = $authenticationUtils->getLastUsername();
+
          return $this->render('login/index.html.twig', [
-             'controller_name' => 'LoginController',
+             'last_username' => $lastUsername,
+             'error'         => $error,
          ]);
      }
  }
```

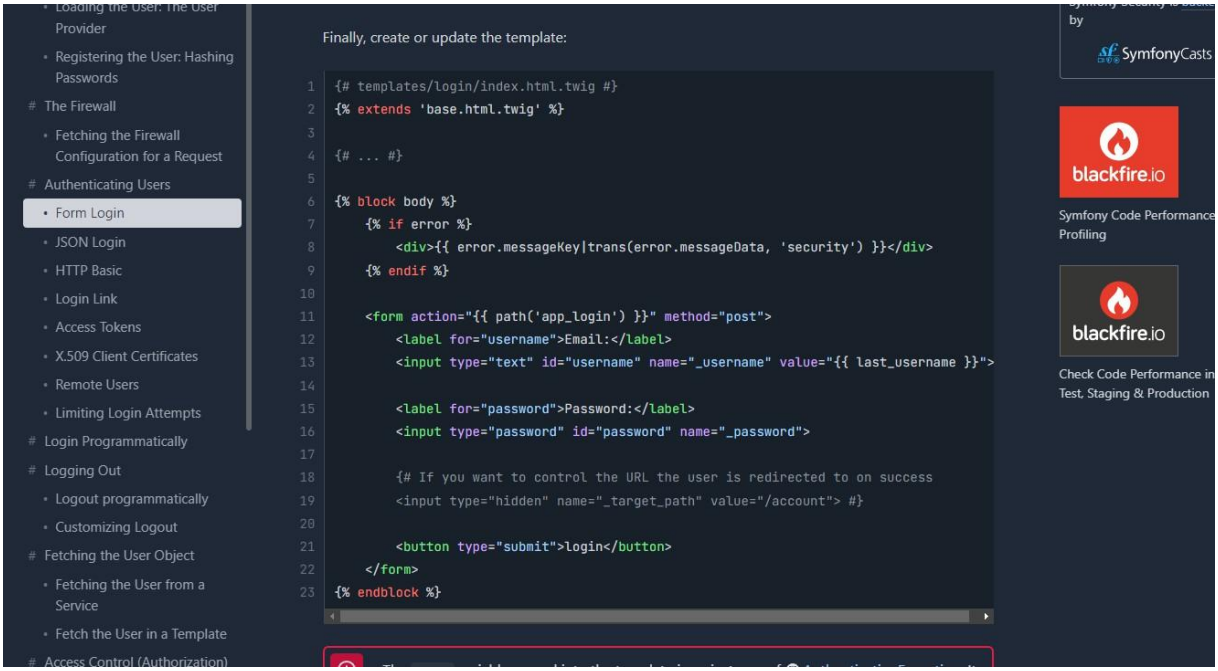
Don't let this controller confuse you. Its job is only to *render* the form: the `form_login` authenticator

On nous dit que la public funtion index est modifiée et que le 'controller_name' est supprimé, par contre on ajoute ce qui est en vert.

Nous allons utiliser le AuthenticationUtils ne pas l'oublié dans les use car il nous permet d'obtenir les methods error et getLastusername

```
3   namespace App\Controller;
4
5   use Symfony\Component\HttpFoundation\Response;
6   use Symfony\Component\Routing\Annotation\Route;
7   use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
8   use Symfony\Component\Security\Http\Authentication\AuthenticationUtils;
9
10  class LoginController extends AbstractController
11  {
12      #[Route('/login', name: 'app_login')]
13      public function index(AuthenticationUtils $authenticationUtils): Response
14      {
15          // get the login error if there is one
16          $error = $authenticationUtils->getLastAuthenticationError();
17          // last username entered by the user
18          $lastUsername = $authenticationUtils->getLastUsername();
19
20          return $this->render('login/index.html.twig', [
21              'last_username' => $lastUsername,
22              'error'         => $error,
23          ]);
24      }
25  }
26
```

Toujours avec la documentation de symfony on nous dit de créer le template dans notre fichier index.html.twig du dossier login. Souvenez vous il a été créé auparavant donc retournons dans notre fichier et recopions le code proposé dans la documentation




```php
namespace App\Controller;

use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\Security\Http\Authentication\AuthenticationUtils;

class LoginController extends AbstractController
{
    #[Route('/login', name: 'app_login')]
    public function index(AuthenticationUtils $authenticationUtils): Response
    {
        // get the login error if there is one
        $error = $authenticationUtils->getLastAuthenticationError();
        // last username entered by the user
        $lastUsername = $authenticationUtils->getLastUsername();

        return $this->render('pages/login/index.html.twig', [
            'last_username' => $lastUsername,
            'error'         => $error,
        ]);
    }
}
```

Regardons dans notre navigateur en ajoutant dans l'url « /login »

Nous avons effectivement un formulaire il nous manque un titre pour cette page et la mise en page du formulaire n'est pas extraordinaire ce sera bien évidemment à nous de le faire.

Mettons en page donc ce formulaire

- On ajoute un titre
- On centre nos label et nos inputs ainsi que notre bouton

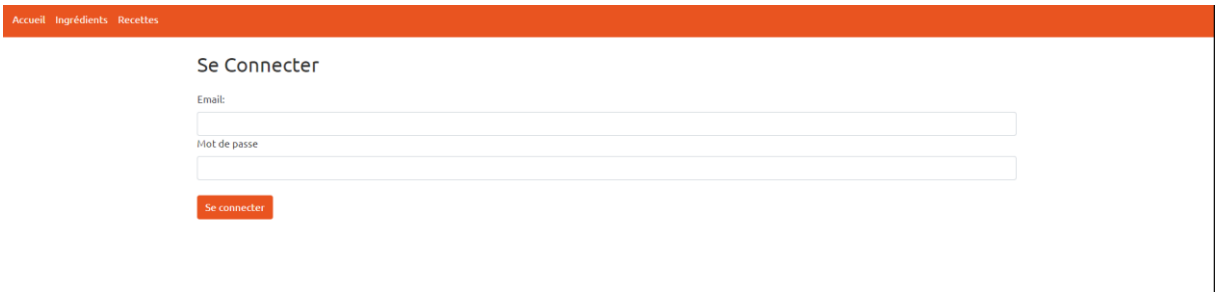
```
11
12      <form action="{{ path('app_login') }}" method="post" class="mt-4">
13          <div>
14              <div>
15                  <label for="username" class="form-label">Email:</label>
16              </div>
17              <div>
18                  <input class="form-control" type="text" id="username" name="_username" value="{{ last_username }}'
19              </div>
20          </div>
21          <div>
22              <div>
23                  <label for="password" class="form-label">Mot de passe</label>
24              </div>
25              <div>
26                  <input class="form-control" type="password" id="password" name="_password">
27              </div>
28          </div>
29          {# If you want to control the URL the user is redirected to on success
30          <input type="hidden" name="_target_path" value="/account"> #}
31
32          <button type="submit" class="btn btn-primary mt-4">Se connecter</button>
33      </form>
```

Le code ci-dessus est pour le formulaire

Voyons pour le titre

```
7       <h2 class="mt-4">Se Connecter</h2>
8       {% if error %}
9           <div>{{ error.messageKey|trans(error.me
10      {% endif %}
11
12      <form action="{{ path('app_login') }}" meth
```

Et je mets le code du block body dans un container « <div class= 'container' » pour avoir ce rendu sur le navigateur



Modifions notre décorateur dans notre controller et ajoutons les methods

```
11  {
12      #[Route('/login', name: 'app_login', methods:['GET','POST'])]
13      public function index(AuthenticationUtils $authenticationUtils): Response
```

Essayons de nous connecter prenons un mail d'un de nos user de notre bdd (exemple dans ma bdd j'ai : remy.guillaume@rousset.com)

Nous savons même si le mdp est encodé que le mdp est 'password' car nous l'avions défini

Regardons ce que cela nous affiche

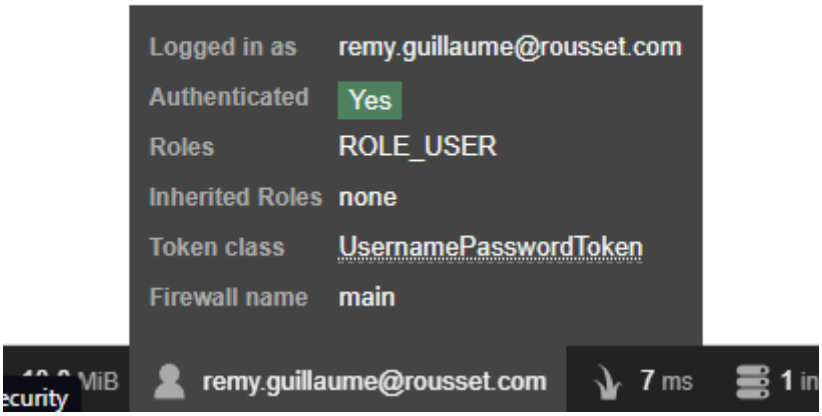
- Nous revenons sur le home
- Nous sommes connecté sur ce compte

Et pour plus de détails :

Nous avons l'adresse mail

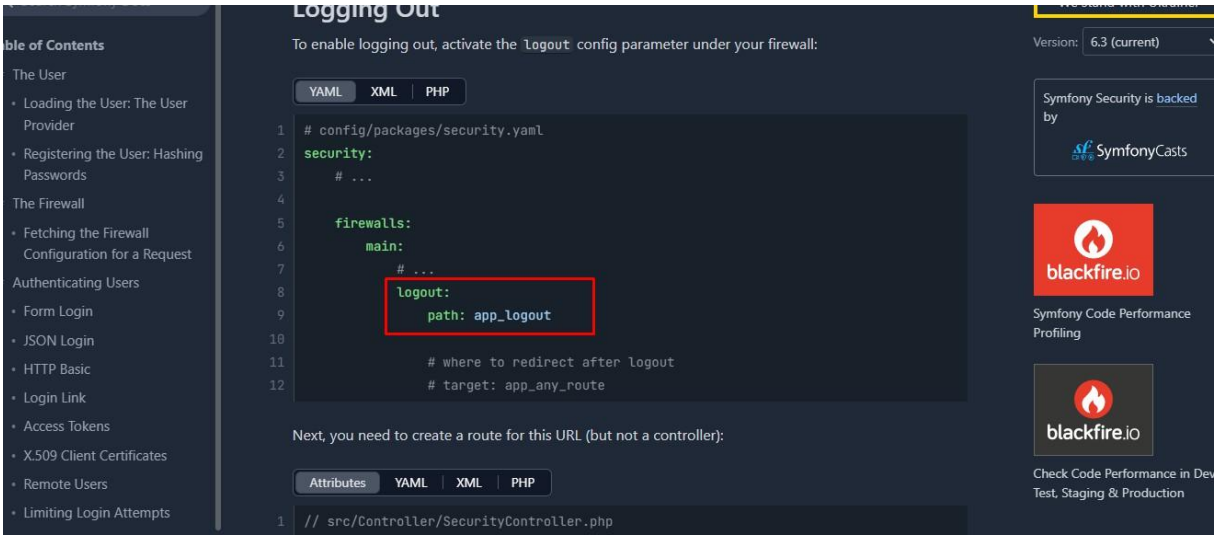
Son Rôle

Son Authentification etc…

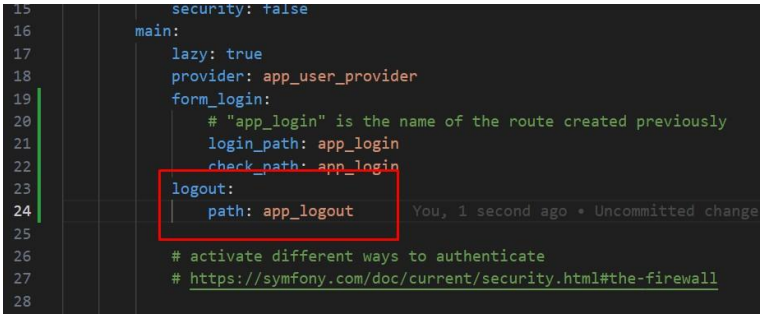


Notre formulaire fonctionne très bien, nous allons maintenant faire un moyen de déconnexion en suivant la [documentation]

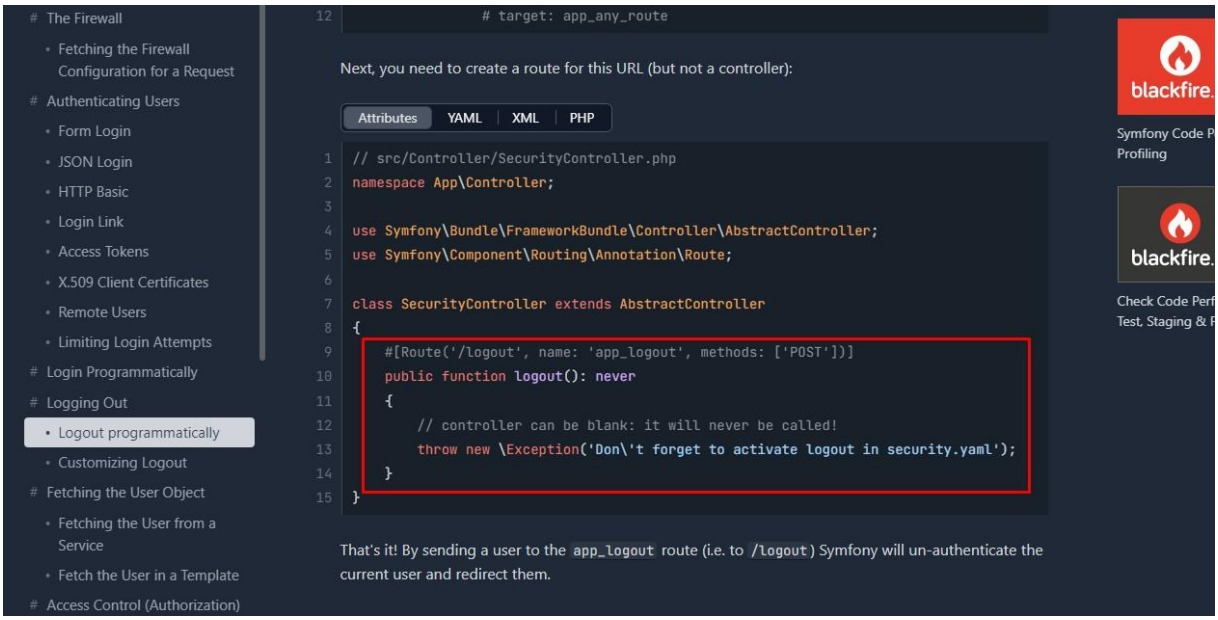
- Dans le sécurity.yaml au niveau toujours des firewalls dans le main il faut rajouter le logout avec son chemin



Donc Dans notre fichier cela donne :



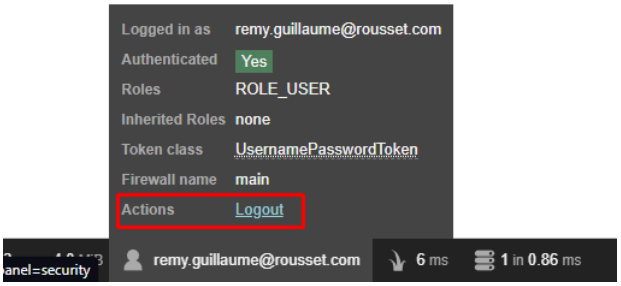
Dans notre LoginController nous devons faire notre method logout comme précisé dans la documentaion :



Allons mettre ce code dans notre controller LoginController de la façon suivante

```
27    #[Route('/deconnexion', name: 'app_logout')]
28    public function logout(): never
29    {
30        // controller can be blank: it will never be called!
31        throw new Exception('Don\'t forget to activate logout in security.yaml');
32    }
```

Regardons sur le navigateur ce qui s'est rajouté



Nous avons le logout qui s'est rajouté et si nous cliquons dessus nous revenons en anonyme

Toutefois si un utilisateur rentre un identifiant ou un mot de passe non valid il faut que le message d'erreur soit bien visible.

Dans notre fichier index.html.twig de notre dossier login nous allons reprendre le code d'une alert de bootswatch et mettre dans cette alert le message d'erreur comme ci-dessous

```
6   <div class="container">
7       <h2 class="mt-4">Se Connecter</h2>
8       {% if error %}
9       <div class="alert alert-dismissible alert-danger">
10          {{ error.messageKey|trans(error.messageData, 'security') }}
11      </div>
12
13      {% endif %}
14
```

Et essayons de mettre un user qui n'est pas dans notre bdd et regardons ce que cela nous retourne dans le navigateur

Parfait nous avons bien notre alert qui revient comme nous le souhaitions

C'est tout pour ce chapitre nous avons réussi à faire notre connexion sans difficultés majeures, la prochaine étape ce sera le formulaire d'inscriptions avec la sécurité